

Rapport de Stage – Développement d'Outils de Gestion

Titre (Anglais) : Development of Management Tools

Stagiaire : Maxence Bonnet

Formation : 1ère Année Ingénieur – Informatique et Cybersécurité (ICS)

École : CPE Lyon

Année Universitaire : 2024—2025

Entreprise d'accueil : Natécia (Groupe Noalys)

Maître de stage : Juliette Durousset

Période : 2 Septembre 2024 – 27 Septembre 2024

Destinataires :

- Maître de stage : Juliette Durousset
- Tuteur école : Juliette Durousset
- École : CPE Lyon

Mots-clés : Développement applicatif, Flutter, Python, Automatisation de workflow, Intelligence Artificielle



Résumé

Ce rapport présente les travaux réalisés durant mon stage de première année au sein de la clinique Natécia (Groupe Noalys). L'objectif principal de ce stage était d'améliorer et de fluidifier les processus internes de gestion des notes de frais et des demandes de remboursement par la conception et le développement de trois applications logicielles distinctes.

Face à des procédures manuelles chronophages et sources d'erreurs, la mission a consisté à :

1. **Développer une application mobile multi-plateforme (iOS/Android) en Flutter/Dart** pour permettre aux employés de soumettre leurs notes de frais de manière dématérialisée. L'application intègre une analyse par Intelligence Artificielle (Google Gemini) pour l'extraction automatique des données depuis les justificatifs.
2. **Adapter cette application pour les dirigeants (PDG)**, avec des fonctionnalités avancées, notamment l'export direct des données vers Google Sheets pour un suivi comptable optimisé.
3. **Créer une application de bureau en Python** pour la gestion centralisée du workflow de validation des demandes de remboursement de trop-perçus, impliquant plusieurs services (facturation, comptabilité, direction).

Ce rapport détaille la démarche adoptée pour chaque projet, de l'analyse du besoin à la livraison des solutions fonctionnelles. Il expose les choix architecturaux et technologiques (Flutter, Python, SQLite), les défis rencontrés – notamment l'apprentissage en autonomie de nouvelles technologies et l'intégration d'une base de données sur un lecteur réseau – et les résultats obtenus. Les solutions développées ont permis de moderniser les processus, d'améliorer la traçabilité et de réduire significativement le temps de traitement administratif.

Sommaire

Résumé

Introduction

Présentation du Stagiaire

Présentation de l'Entreprise : Natécia

Contexte et Enjeux du Stage

Structure du Rapport

Application Mobile "Notes de Frais Employé"

Analyse du Besoin

Démarche Adoptée et Solutions Envisagées

Justification des Choix Techniques

Fonctionnement de l'Application

- Workflow Principal
- Architecture C4 - Contexte
- Architecture C4 - Conteneurs
- Interface Utilisateur

Fonctions Clés Issues du Code

Difficultés Rencontrées

Résultats Obtenus

Application Mobile "Notes de Frais PDG"

Analyse du Besoin Spécifique

Adaptations et Fonctionnalités Clés

- Interface PDG Spécialisée

Résultats

Application Desktop "Gestion des Remboursements"

Analyse du Besoin

Architecture Logicielle MVC

- Schéma d'Architecture
- Diagramme C4 - Contexte
- Diagramme C4 - Conteneurs

Fonctions Clés Issues du Code

Fonctionnement et Workflow de Validation

- Diagramme de Workflow
- Interface Desktop

Gestion de la Base de Données sur Réseau

Résultats Obtenus

1. Introduction

1.1 Présentation du Stagiaire

Récapitulatif des Livrables

Je suis Maxence Bonnet, un étudiant de 22 ans en première année de cycle ingénieur en Informatique, spécialisé en Intelligence Cognitive (ICS) à CPE Lyon. Mon parcours académique a débuté par un Baccalauréat STI2D spécialité SIN (Systèmes d'Information et Numérique), suivi d'un BTS SNIR (Systèmes Numériques, option Informatique et Réseaux). Cette formation m'a permis d'acquérir de solides compétences en développement logiciel et en administration système, que j'ai souhaité mettre en pratique dans un contexte professionnel concret.

1.2 Présentation de l'Entreprise : Natécia

Bibliographie

Natécia est une clinique privée lyonnaise spécialisée dans la santé de la femme, de la mère et de l'enfant. En tant que membre du groupe Noalys, elle s'inscrit dans un réseau d'établissements de santé reconnus pour la qualité de leurs soins, complétant l'hôpital privé Natécia. Le stage s'est déroulé au sein du service administratif de la clinique, où un besoin de modernisation des outils informatiques a été identifié pour optimiser les tâches de gestion interne.

Annexes

Liens vers les Dépôts de Code Source

L'hôpital privé Natecia en chiffres			
2	239	18	120
Sites	Salariés	Spécialités	Médecins
17.000	9.123	3781	181
Patients accueillis/an	Interventions/an	Naissances	Lits & places

Capture – Présentation visuelle de Natécia (page d'accueil du site ou bâtiment avec logo).

1.3 Contexte et Enjeux du Stage

Le point de départ de ce stage était un constat simple : les processus de gestion des notes de frais et des demandes de remboursement étaient majoritairement manuels, reposant sur des échanges d'e-mails et des documents papier. Cette méthode de travail présentait plusieurs inconvénients majeurs :

- Perte de temps pour les employés et les services comptables.
- Risques d'erreurs de saisie et de perte de justificatifs.
- Manque de traçabilité et de vision centralisée sur l'état des demandes.
- Processus de validation long et complexe, impliquant de multiples intervenants.
- Impact économique important bloquant les demandes en cours.

L'objectif du stage était donc de répondre à ces problématiques en développant des solutions logicielles sur mesure, capables d'automatiser et de sécuriser ces workflows.

1.4 Structure du Rapport

Ce rapport s'articule en trois chapitres principaux, chacun dédié à l'un des projets développés. Nous commencerons par les deux applications mobiles de notes de frais, puis nous aborderons l'application de bureau pour la gestion des remboursements. Chaque chapitre décrira la démarche suivie, les choix techniques effectués et les résultats obtenus. Enfin, une conclusion générale dressera le bilan du stage, ses apports et les perspectives d'évolution des outils créés.

2. Application Mobile "Notes de Frais Employé"

2.1 Analyse du Besoin

Le premier projet visait à dématérialiser entièrement le processus de soumission des notes de frais pour les collaborateurs de Natécia. Le cahier des charges fonctionnel était le suivant :

- Permettre la capture de justificatifs via l'appareil photo ou l'import de fichiers (images, PDF).
- Extraire automatiquement les informations clés (marchand, date, montant, TVA) grâce à une IA pour automatiser la saisie de remboursement des documents, la gestion des notes de frais étant auparavant précaires.
- Permettre à l'utilisateur de vérifier et corriger les données extraites.
- Gérer les notes de frais kilométriques.
- Conserver un historique des notes soumises et en attente.
- Générer un rapport PDF et l'envoyer par e-mail au service comptable avec les justificatifs en pièces jointes.
- Fonctionner sur iOS et Android.

2.2 Démarche Adoptée et Solutions Envisagées

Face au besoin d'une application multi-plateforme, le choix s'est naturellement porté sur un framework de développement cross-platform. Après étude, **Flutter** a été retenu pour sa performance, son écosystème riche et sa facilité de prise en main.

Pour l'extraction de données, la solution d'une **API d'Intelligence Artificielle** a été privilégiée par rapport à une simple technologie OCR, afin d'obtenir une analyse sémantique du document (catégorisation de la dépense) et un indice de confiance sur les données extraites.

2.3 Justification des Choix Techniques

- **Langage** : Dart (via Flutter) pour le développement mobile.
- **Framework UI** : Flutter pour sa capacité à produire des interfaces natives compilées pour iOS et Android à partir d'une seule base de code.
- **Gestion de l'état** : **Riverpod**, pour une gestion d'état réactive, robuste et scalable.
- **Analyse IA** : API **Google Gemini**, pour ses capacités avancées en analyse de documents et extraction d'informations structurées.
- **Stockage local** : Base de données NoSQL **Hive**, très performante et bien intégrée à l'écosystème Flutter, pour stocker l'historique des notes de frais.
- **Envoi d'e-mails** : Bibliothèque **mailer** pour l'envoi des rapports en tâche de fond.

2.4 Fonctionnement de l'Application

L'application suit un workflow utilisateur simple et intuitif.

Figure 1 : Schéma du workflow de l'application "Notes de Frais"

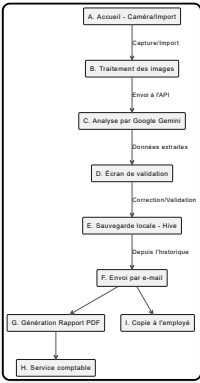


Figure 1 : Schéma du workflow de l'application "Notes de Frais"

Le workflow ci-dessus illustre le parcours d'un justificatif, de sa capture à sa réception par le service comptable.

Figure 1.1 : Diagramme de Contexte C4 de l'application "Notes de Frais"

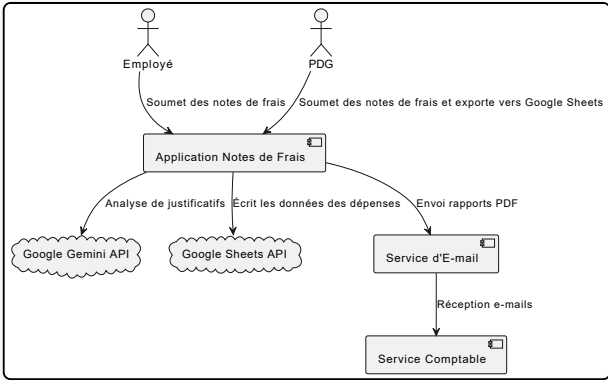


Figure 1.1 : Diagramme de Contexte C4 de l'application "Notes de Frais"

Figure 1.2 : Diagramme de Conteneurs C4 de l'application "Notes de Frais"

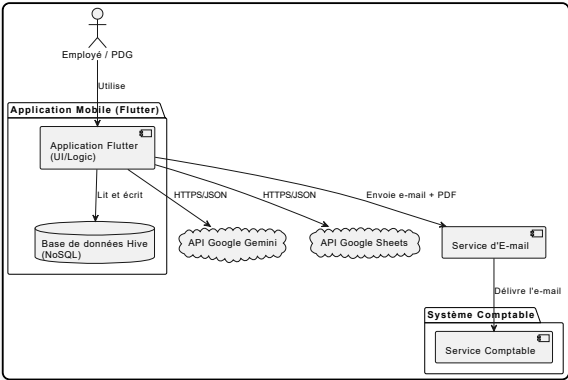
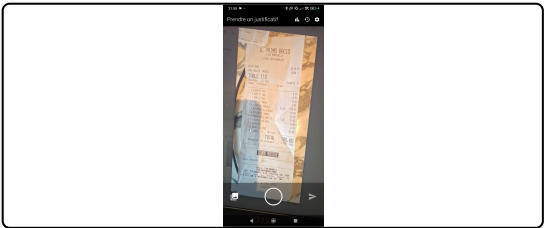
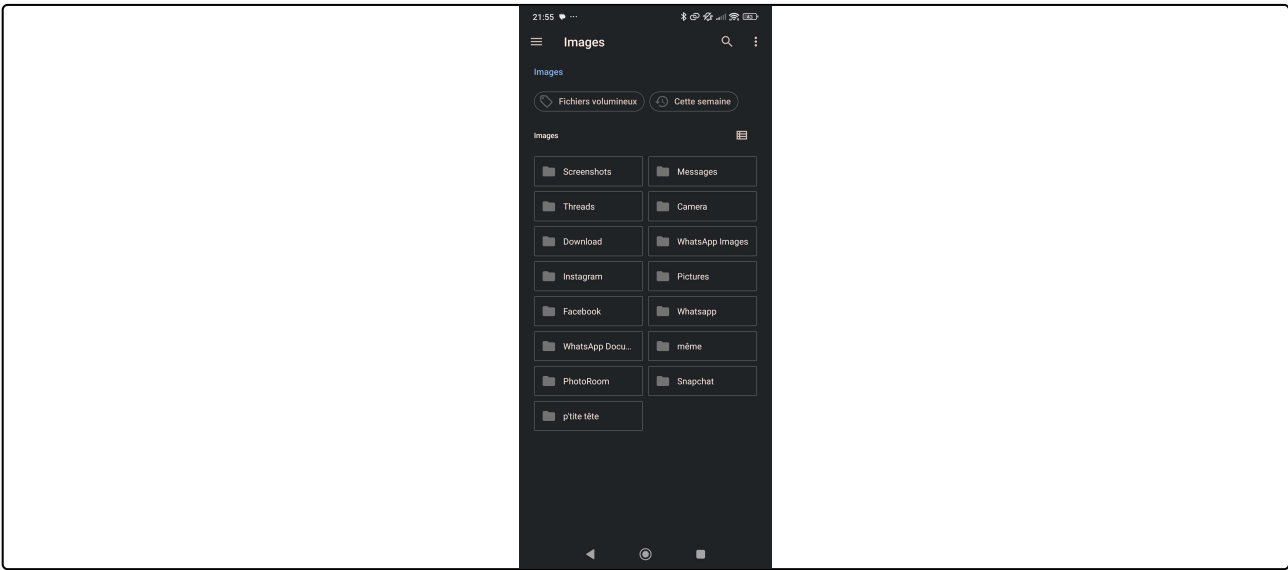


Figure 1.2 : Diagramme de Conteneurs C4 de l'application "Notes de Frais"

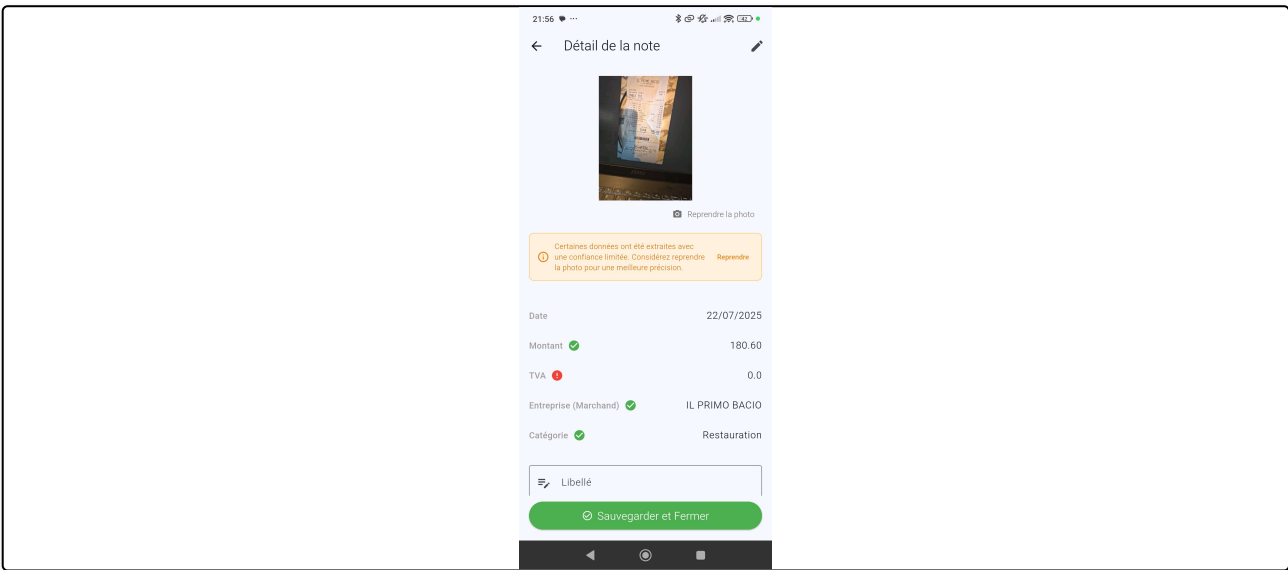


Interface de l'application "Notes de Frais"

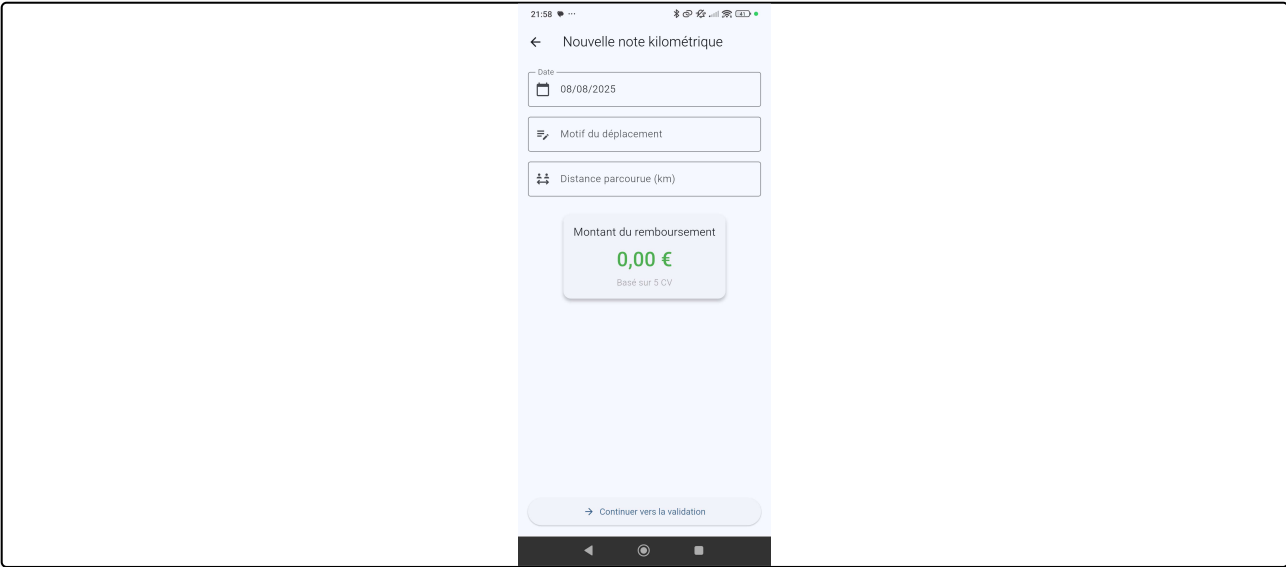
Captures complémentaires - Application Employé



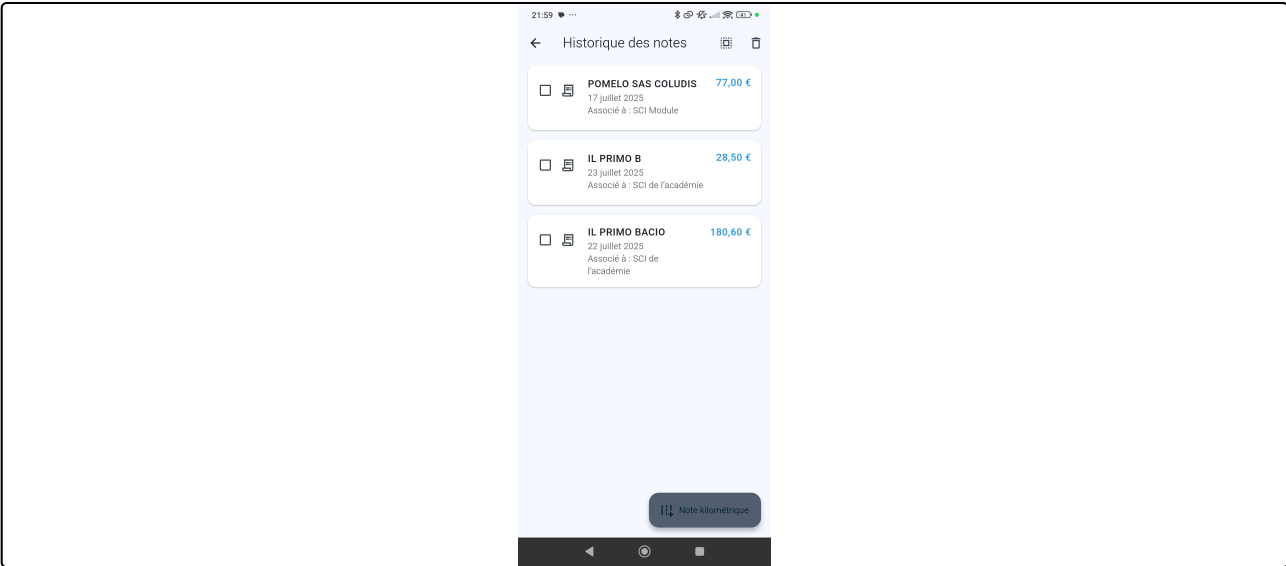
Capture – Écran de capture/import de justificatifs (caméra ou sélecteur multi-fichiers).



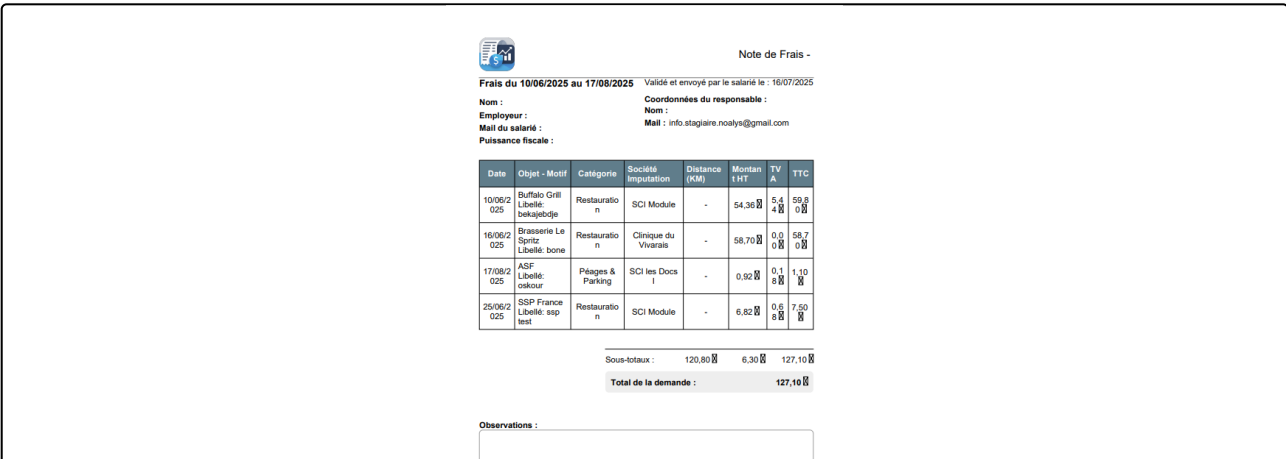
Capture – Analyse IA et vérification/correction des données extraites (marchand, date, montant, TVA, indicateurs de confiance).

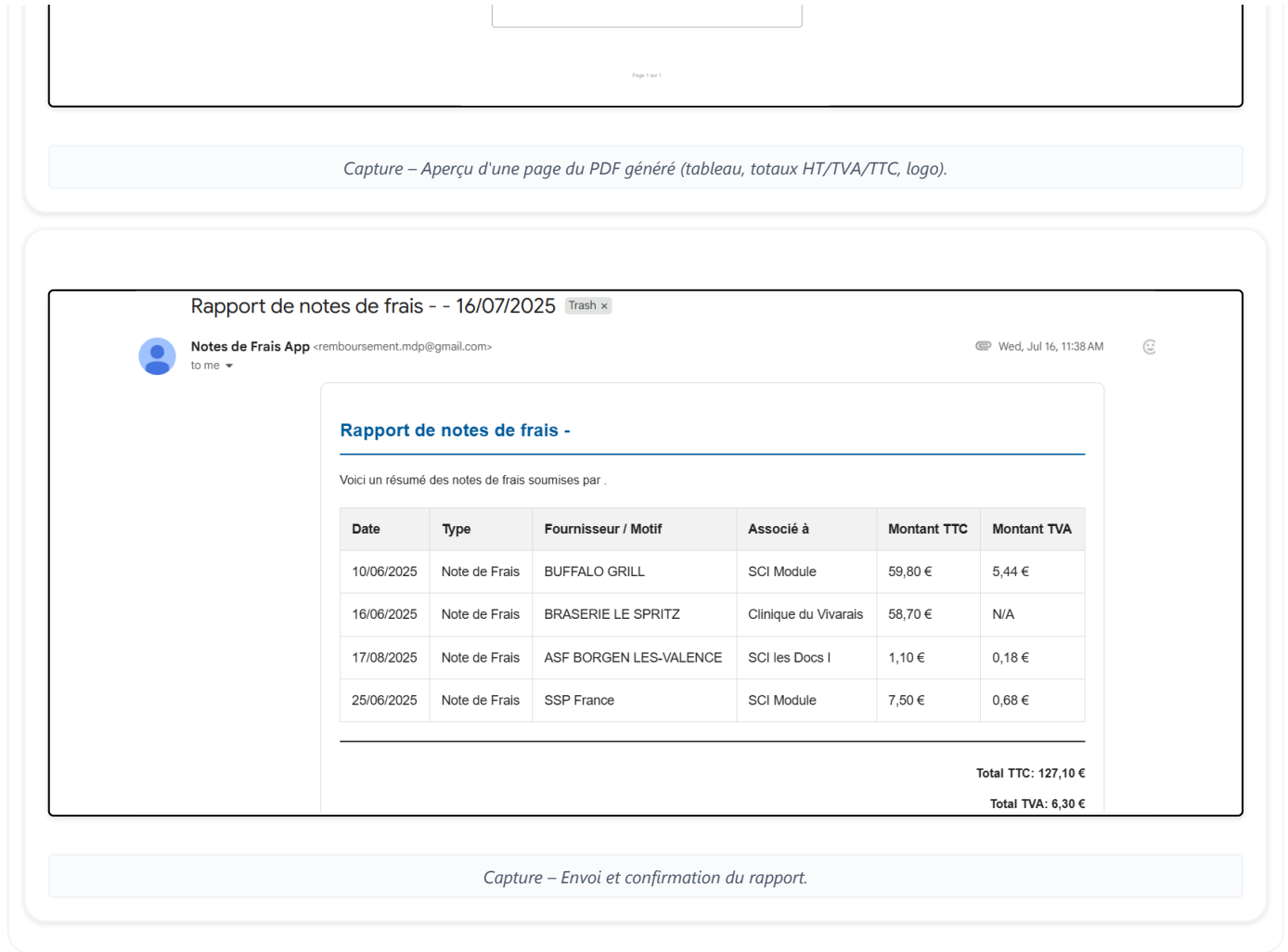


Capture – Notes kilométriques (distance, barème, calcul et récapitulatif).



Capture – Historique des notes (statuts envoyée/en attente, filtres).





2.5 Fonctions Clés Issues du Code

- `lib/services/ai_service.dart::extractExpenseDataFromFiles` : appel à Google Gemini avec prompt structuré, extraction JSON et indices de confiance.
- `lib/services/pdf_service.dart::generateExpenseReportPdf` : génération d'un PDF A4 (tables, totaux HT/TVA/TTC, polices embarquées, logo, sections d'observations).
- `lib/services/email_service.dart::sendExpenseBatchEmail` : envoi HTML avec PJ (PDF récap + justificatifs renommés), objet et CC salarié.
- `lib/services/background_task_service.dart::processQueue` : écoute réseau, reprise des envois, nettoyage fichiers, marquage `isSent`.
- `lib/views/camera_view.dart` : cycle de vie caméra, capture/import multi-fichiers, navigation vers traitement.

2.6 Difficultés Rencontrées

La principale difficulté de ce projet a été **l'apprentissage en totale autonomie du framework Flutter et du langage Dart**. Cela a nécessité un investissement initial important pour comprendre les concepts de widgets, de gestion d'état et l'écosystème de publication.

Une autre difficulté a été la gestion du **traitement asynchrone** pour l'envoi des e-mails, afin que l'application reste fluide et puisse gérer les envois même en cas de connectivité réseau intermittente.

2.7 Résultats Obtenus

L'application développée est fonctionnelle et répond à toutes les exigences du cahier des charges. Elle a été testée par plusieurs utilisateurs (Benoit Gonnet, Philippe NERI) qui ont salué sa simplicité d'utilisation et le gain de temps réalisé. Le processus de soumission d'une note de frais, qui prenait auparavant plusieurs minutes de saisie manuelle, est désormais réalisable en moins d'une minute. Les justificatifs ne sont plus perdus et peuvent être rapidement consultés dans un format dématérialisé, simplifiant son archivage.

3. Application Mobile "Notes de Frais PDG"

3.1 Analyse du Besoin Spécifique

Une version de l'application a été spécifiquement demandée pour les dirigeants de l'entreprise. Bien que partageant 90% des fonctionnalités de la version "Employé", elle devait intégrer une fonctionnalité clé supplémentaire :

[l'export des données vers une feuille de calcul Google Sheets partagée](#)
, en plus de l'envoi par e-mail.

L'objectif était de permettre un suivi financier en temps réel et de faciliter l'agrégation des données à des fins de reporting.

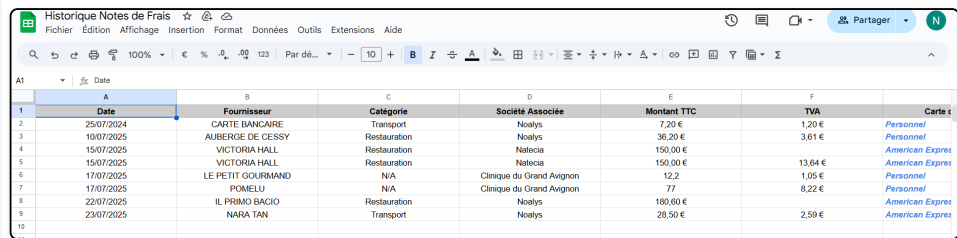
3.2 Adaptations et Fonctionnalités Clés

Le projet a donc consisté à dupliquer la base de code existante (via une branche Git distincte, `Switch`) et à y intégrer l'API Google Sheets.

- **Authentification Google** : Mise en place d'un flux d'authentification OAuth 2.0 pour permettre à l'application d'accéder aux services Google au nom de l'utilisateur.
- **API Google Sheets** : Utilisation des bibliothèques `googleapis` et `googleapis_auth` pour écrire une nouvelle ligne dans un Google Sheet spécifié à chaque note de frais validée.

Le reste de l'application (capture, analyse IA, stockage local) est resté identique, garantissant une maintenance aisée des deux versions.

Captures d'écran - Application PDG



	A	B	C	D	E	F	G
1	Date	Fournisseur	Catégorie	Société Associée	Montant TTC	TVA	Carte
2	25/07/2024	CARTE BANCAIRE	Transport	Noalys	7,20 €	1,20 €	Personnel
3	10/07/2025	AUBERGE DE CESSY	Restauration	Noalys	36,20 €	3,61 €	Personnel
4	15/07/2025	VICTORIA HALL	Restauration	Natecia	150,00 €		American Express
5	15/07/2025	VICTORIA HALL	Restauration	Natecia	150,00 €	13,64 €	American Express
6	17/07/2025	LE PETIT GOURMAND	N/A	Clinique du Grand Avignon	12,2	1,05 €	Personnel
7	17/07/2025	POMELU	N/A	Clinique du Grand Avignon	77	8,22 €	Personnel
8	22/07/2025	IL PRIMO BACCIO	Restauration	Noalys	180,80 €		American Express
9	23/07/2025	NARA TAN	Transport	Noalys	28,50 €	2,50 €	American Express
10							

Capture – Résultat d'export : ligne ajoutée dans Google Sheets (toutes les colonnes pertinentes renseignées).

3.3 Résultats

La version "PDG" est également fonctionnelle. L'intégration avec Google Sheets apporte une plus-value significative en termes de suivi et d'analyse des dépenses, offrant une vision consolidée et instantanée qui n'existait pas auparavant.

4. Application Desktop "Gestion des Remboursements"

4.1 Analyse du Besoin

Le troisième projet adressait une problématique différente : la gestion des demandes de remboursement de trop-perçus clients. Ce processus, critique pour la satisfaction client et la rigueur comptable, était entièrement géré par e-mail, ce qui entraînait des retards et un manque de visibilité. Cette principale problématique - l'absence de visibilité de la demande en ne pouvant pas savoir où en était son demande - devait être adressée par l'outil.

Le besoin était de créer une application de bureau centralisée, avec :

- Une gestion des utilisateurs basée sur des rôles (Demandeur, Comptable, Validateur, etc.).
- Un workflow de validation en plusieurs étapes.
- La possibilité de joindre des documents (RIB, facture).
- Un historique complet et immuable de chaque demande.
- Une base de données unique, partagée et accessible depuis un lecteur réseau.

4.2 Architecture Logicielle MVC

Pour garantir la maintenabilité et la scalabilité de l'application, une architecture **Modèle-Vue-Contrôleur (MVC)** a été mise en œuvre.

Figure 2 : Schéma d'architecture de l'application "Gestion des Remboursements"

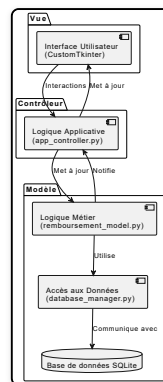


Figure 2 : Schéma d'architecture de l'application "Gestion des Remboursements"

* **Modèle :** Gère les données et la logique métier. Il interagit directement avec la base de données SQLite. * **Vue :** Responsable de l'affichage de l'interface graphique. Elle est construite avec la bibliothèque Python `CustomTkinter`. * **Contrôleur :** Reçoit les actions de l'utilisateur depuis la Vue, les traite en faisant appel au Modèle, et met à jour la Vue en conséquence.

Figure 2.1 : Diagramme de Contexte C4 de l'application "Gestion des Remboursements"

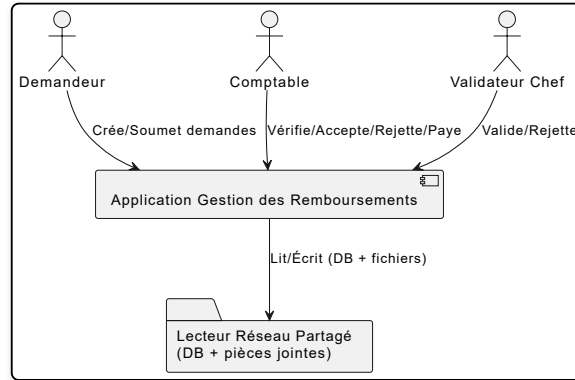


Figure 2.1 : Diagramme de Contexte C4 de l'application "Gestion des Remboursements"

Figure 2.2 : Diagramme de Conteneurs C4 de l'application "Gestion des Remboursements"

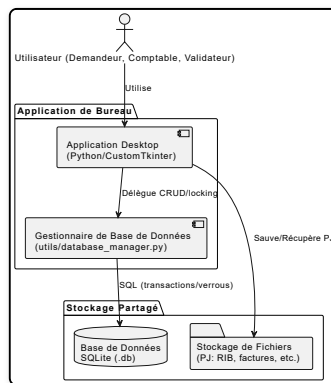


Figure 2.2 : Diagramme de Conteneurs C4 de l'application "Gestion des Remboursements"

4.3 Fonctions Clés Issues du Code

- `models/remboursement_model.py::creer_nouvelle_demande` : création demande, organisation des pièces (RIB/Facture/TP), historique initial.
- `models/remboursement_workflow.py` : transitions (accepter/refuser constat, valider/refuser, confirmer paiement, resoumission) avec historisation.
- `models/remboursement_data.py::charger_demandes_data` : filtrage multi-critères, pagination, reconstruction modèle + historiques + PJ.
- `controllers/remboursement_controller.py` : pont UI/métier, copie versionnée des PJ, extraction d'archive ZIP à la volée, actions admin.
- `controllers/app_controller.py` : cycle de vie app, cache utilisateurs, tâches de démarrage (archivage, sauvegarde), bannière réseau, vérifs BDD.

4.4 Fonctionnement et Workflow de Validation

L'application matérialise un processus métier strict où une demande de remboursement passe par plusieurs statuts, chaque transition étant conditionnée par l'action d'un utilisateur avec le rôle approprié.

Figure 3 : Diagramme du workflow de validation des remboursements

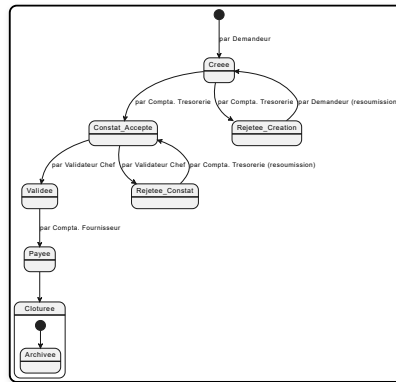
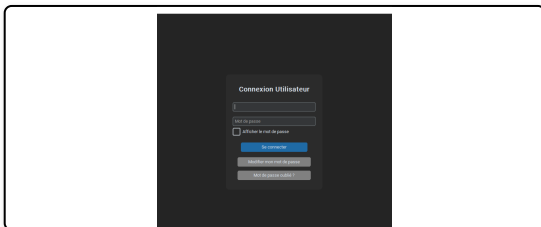


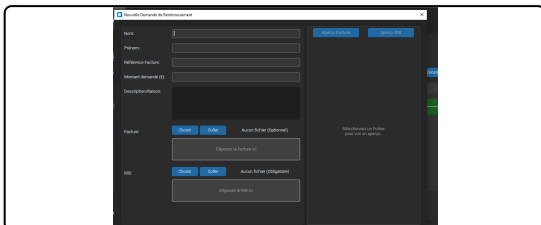
Figure 3 : Diagramme du workflow de validation des remboursements

Ce diagramme illustre les différents états d'une demande et les rôles habilités à la faire transiter d'un état à l'autre.

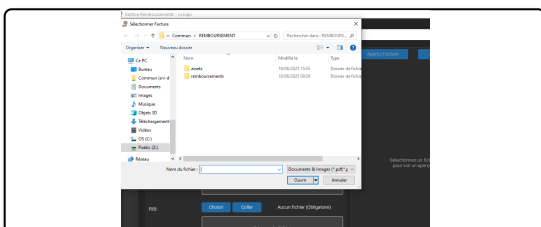
Captures complémentaires - Application Desktop



Capture – Écran de connexion (identifiant, mot de passe, messages d'erreur possibles).



Capture – Création d'une nouvelle demande (identité, montant, motif).



Capture – Gestion des pièces jointes (RIB, facture, justificatifs).



4.5 Gestion de la Base de Données sur Réseau

Une contrainte forte du projet était l'utilisation d'une base de données **SQLite locale, mais située sur un disque réseau partagé** pour être accessible par tous les utilisateurs. Cette contrainte a introduit des défis de performance et de concurrence d'accès. Des mécanismes de **verrouillage de fichier** (file locking) et une gestion rigoureuse des transactions ont été mis en place dans le **database_manager.py** pour éviter la corruption de la base de données lorsque plusieurs utilisateurs effectuent des opérations simultanément.

4.6 Résultats Obtenus

L'application a été déployée et est utilisée par les différents services concernés (Marie LUPO, Pascale DIOP). Elle a permis de :

- Standardiser le processus de demande de remboursement.
- Réduire de plus de 50% le temps de traitement moyen d'une demande.
- Garantir une traçabilité complète de toutes les actions.
- Centraliser tous les documents et informations en un seul point.

5. Conclusion

5.1 Récapitulatif des Livrables

Ce stage a abouti à la livraison de trois applications entièrement fonctionnelles qui répondent aux problématiques de gestion initiales de Natécia :

1. Une application mobile "Notes de Frais" pour les employés.
2. Une version "PDG" de cette application avec des fonctionnalités de reporting étendues.
3. Une application de bureau "Gestion des Remboursements" pour centraliser et sécuriser le workflow de validation.

Ces outils ont été conçus pour être robustes, sécurisés et évolutifs.

5.2 Limites et Perspectives d'Évolution

Bien que fonctionnels, les projets présentent des axes d'amélioration :

- **Synchronisation Cloud** : Pour les applications mobiles, une synchronisation des données (historique) sur un service Cloud permettrait à l'utilisateur de retrouver ses données en changeant de téléphone.
- **Déploiement centralisé** : Pour l'application de bureau, un mécanisme de mise à jour automatique simplifierait la maintenance.
- **Tests Unitaires** : Le développement de suites de tests unitaires et d'intégration plus complètes permettrait de renforcer la fiabilité du code.
- **Serveur distant** : À terme, la migration de la base de données SQLite vers un vrai serveur de base de données (PostgreSQL, MySQL) serait bénéfique pour les performances et la scalabilité.

Remerciements

Je tiens à exprimer ma profonde gratitude à ma maître de stage,

Mme Juliette Durousset

, pour sa confiance, sa disponibilité et ses précieux conseils tout au long de cette période.

Je remercie également l'ensemble des collaborateurs qui ont contribué au succès de ces projets par leurs retours et leur participation aux phases de test, notamment

M. Benoit Gonnet

,

M. Philippe NERI

,

Mme Pascale DIOP

et

Mme Marie LUPO

.

Enfin, je remercie CPE Lyon de m'avoir offert l'opportunité de réaliser ce stage enrichissant qui a été une excellente introduction au monde professionnel de l'ingénierie logicielle.

Glossaire

API :Application Programming Interface. Interface de programmation applicative.

Flutter :Framework de développement d'interface utilisateur open-source créé par Google.

Dart :Langage de programmation optimisé pour le développement d'applications sur de multiples plateformes.

Gemini :Famille de modèles d'IA multimodaux développée par Google.

Hive :Base de données NoSQL clé-valeur, légère et rapide, écrite en Dart.

MVC :Modèle-Vue-Contrôleur. Patron d'architecture logicielle.

OCR :Optical Character Recognition. Reconnaissance optique de caractères.

SQLite :Système de gestion de base de données relationnelle contenu dans une bibliothèque C.

Bibliographie

- Documentation officielle de Flutter : <https://flutter.dev/docs>
 - Documentation de l'API Google Gemini : <https://ai.google.dev/docs>
 - Documentation de la bibliothèque Python CustomTkinter : <https://customtkinter.com/documentation>
 - Documentation de la base de données Hive pour Flutter : <https://docs.hivedb.dev/>
-

Annexes

Liens vers les dépôts de code source

- **Projet Notes de Frais (Employé & PDG)** : https://github.com/Maxenss-Bonnet/notes_de_frais
 - Branche Employé : [Employer](#)
 - Branche PDG : [Switch](#)
- **Projet Gestion des Remboursements** : <https://github.com/Maxenss-Bonnet/Remboursement/tree/Main>